PEP 523 – Adding a frame evaluation API to CPython

Author: Brett Cannon <brett at python.org>, Dino Viehland <dinoviehland at gmail.com>

Status: Final
Type: Standards Track
Created: 16-May-2016
Python-Version: 3.6
Post-History: 16-May-2016
Resolution: Python-Dev message (https://mail.python.org/pipermail/python-dev/2016-August/145937.html)

Source: https://github.com/python/peps/blob/main/peps/pep-0523.rst

Last modified 2026-01-13 17:42:26 UTC

Abstract (#abstract)

This PEP proposes to expand CPython's C API [2] (#c-api) to allow for the specification of a per-interpreter function pointer to handle the evaluation of frames [5] (#pyeval-evalframeex). This proposal also suggests adding a new field to code objects [3] (#pycodeobject) to store arbitrary data for use by the frame evaluation function.

Rationale (#rationale)

One place where flexibility has been lacking in Python is in the direct execution of Python code. While CPython's C API [2] (#c-api) allows for constructing the data going into a frame object and then evaluating it via `PyEval_EvalFrameEx()` [5] (#pyeval-evalframeex), control over the execution of Python code comes down to individual objects instead of a holistic control of execution at the frame level.

While wanting to have influence over frame evaluation may seem a bit too low-level, it does open the possibility for things such as a method-level JIT to be introduced into CPython without CPython itself having to provide one. By allowing external C code to control frame evaluation, a JIT can participate in the execution of Python code at the key point where evaluation occurs. This then allows for a JIT to conditionally recompile Python bytecode to machine code as desired while still

llowing for executing regular CPython bytecode when running the JIT is not esired. This can be accomplished by allowing interpreters to specify what unction to call to evaluate a frame. And by placing the API at the frame evaluation evel it allows for a complete view of the execution environment of the code for the IT.

his ability to specify a frame evaluation function also allows for other use-cases eyond just opening CPython up to a JIT. For instance, it would not be difficult to nplement a tracing or profiling function at the call level with this API. While CPython does provide the ability to set a tracing or profiling function at the Python evel, this would be able to match the data collection of the profiler and quite ossibly be faster for tracing by simply skipping per-line tracing support.

also opens up the possibility of debugging where the frame evaluation function nly performs special debugging work when it detects it is about to execute a pecific code object. In that instance the bytecode could be theoretically rewritten n-place to inject a breakpoint function call at the proper point for help in ebugging while not having to do a heavy-handed approach as required by `ys.settrace()`.

o help facilitate these use-cases, we are also proposing the adding of a “scratch pace” on code objects via a new field. This will allow per-code object data to be tored with the code object itself for easy retrieval by the frame evaluation unction as necessary. The field itself will simply be a `PyObject *` type so that any ata stored in the field will participate in normal object memory management.

roposal (#proposal)

ll proposed C API changes below will not be part of the stable ABI.

xpanding `PyCodeObject` (#expanding-pycodeobject)

ne field is to be added to the `PyCodeObject` struct [3] (#pycodeobject):

```
typedef struct {
   ...
   void *co_extra;  /* "Scratch space" for the code object. */
} PyCodeObject;
```

he `co_extra` will be `NULL` by default and only filled in as needed. Values stored in ıe field are expected to not be required in order for the code object to function, llowing the loss of the data of the field to be acceptable.

. private API has been introduced to work with the field:

```
PyAPI_FUNC(Py_ssize_t) _PyEval_RequestCodeExtraIndex(freefunc);
PyAPI_FUNC(int) _PyCode_GetExtra(PyObject *code, Py_ssize_t index,
                                 void **extra);
PyAPI_FUNC(int) _PyCode_SetExtra(PyObject *code, Py_ssize_t index,
                                 void *extra);
```

lsers of the field are expected to call `_PyEval_RequestCodeExtraIndex()` to receive what should be considered) an opaque index value to adding data into `co-extra`. Vith that index, users can set data using `_PyCode_SetExtra()` and later retrieve the ata with `_PyCode_GetExtra()`. The API is purposefully listed as private to ommunicate the fact that there are no semantic guarantees of the API between ython releases.

lsing a list and tuple were considered but was found to be less performant, and ith a key use-case being JIT usage the performance consideration won out for sing a custom struct instead of a Python object.

. dict was also considered, but once again performance was more important. Vhile a dict will have constant overhead in looking up data, the overhead for the ommon case of a single object being stored in the data structure leads to a tuple aving better performance characteristics (i.e. iterating a tuple of length 1 is faster ıan the overhead of hashing and looking up an object in a dict).

xpanding `PyInterpreterState` (#expanding-pyinterpreterstate)

he entrypoint for the frame evaluation function is per-interpreter:

```
// Same type signature as PyEval_EvalFrameEx().
typedef PyObject* (*_PyFrameEvalFunction)(PyFrameObject*, int);

typedef struct {
    ...
    _PyFrameEvalFunction eval_frame;
} PyInterpreterState;
```

y default, the `eval_frame` field will be initialized to a function pointer that epresents what `PyEval_EvalFrameEx()` currently is (called `PyEval_EvalFrameDefault()`, discussed later in this PEP). Third-party code may hen set their own frame evaluation function instead to control the execution of ython code. A pointer comparison can be used to detect if the field is set to `PyEval_EvalFrameDefault()` and thus has not been mutated yet.

hanges to `Python/ceval.c` (#changes-to-python-ceval-c)

`yEval_EvalFrameEx()` [5] (#pyeval-evalframeex) as it currently stands will be renamed o `_PyEval_EvalFrameDefault()`. The new `PyEval_EvalFrameEx()` will then become:

```
PyObject *
PyEval_EvalFrameEx(PyFrameObject *frame, int throwflag)
{
    PyThreadState *tstate = PyThreadState_GET();
    return tstate->interp->eval_frame(frame, throwflag);
}
```

his allows third-party code to place themselves directly in the path of Python ode execution while being backwards-compatible with code already using the re-existing C API.

pdating `python-gdb.py` (#updating-python-gdb-py)

he generated `python-gdb.py` file used for Python support in GDB makes some ard-coded assumptions about `PyEval_EvalFrameEx()`, e.g. the names of local ariables. It will need to be updated to work with the proposed changes.

erformance impact (#performance-impact)

s this PEP is proposing an API to add pluggability, performance impact is onsidered only in the case where no third-party code has made any changes.

everal runs of pybench [14] (#pybench) consistently showed no performance cost om the API change alone.

run of the Python benchmark suite [9] (#py-benchmarks) showed no measurable ost in performance.

n terms of memory impact, since there are typically not many CPython nterpreters executing in a single process that means the impact of `co_extra` being dded to `PyCodeObject` is the only worry. According to [8] (#code-object-count), a run f the Python test suite results in about 72,395 code objects being created. On a 4-bit CPU that would result in 579,160 bytes of extra memory being used if all ode objects were alive at once and had nothing set in their `co_extra` fields.

Example Usage (#example-usage)

A JIT for CPython (#a-jit-for-cpython)

Pyjion (#pyjion)

The Pyjion project [1] (#id21) has used this proposed API to implement a JIT for CPython using the CoreCLR's JIT [4] (#coreclr). Each code object has its `co_extra` field set to a `PyjionJittedCode` object which stores four pieces of information:

1. Execution count
2. A boolean representing whether a previous attempt to JIT failed
3. A function pointer to a trampoline (which can be type tracing or not)
4. A void pointer to any JIT-compiled machine code

The frame evaluation function has (roughly) the following algorithm:

```
def eval_frame(frame, throw_flag):
    pyjion_code = frame.code.co_extra
    if not pyjion_code:
        frame.code.co_extra = PyjionJittedCode()
    elif not pyjion_code.jit_failed:
        if not pyjion_code.jit_code:
            return pyjion_code.eval(pyjion_code.jit_code, frame)
        elif pyjion_code.exec_count > 20_000:
            if jit_compile(frame):
                return pyjion_code.eval(pyjion_code.jit_code, frame)
            else:
                pyjion_code.jit_failed = True
    pyjion_code.exec_count += 1
    return _PyEval_EvalFrameDefault(frame, throw_flag)
```

he key point, though, is that all of this work and logic is separate from CPython nd yet with the proposed API changes it is able to provide a JIT that is compliant /ith Python semantics (as of this writing, performance is almost equivalent to :Python without the new API). This means there's nothing technically preventing thers from implementing their own JITs for CPython by utilizing the proposed .PI.

ther JITs (#other-jits)

should be mentioned that the Pyston [10] (#pyston) team was consulted on an arlier version of this PEP that was more JIT-specific and they were not interested ı utilizing the changes proposed because they want control over memory layout ney had no interest in directly supporting CPython itself. An informal discussion /ith a developer on the PyPy [11] (#pypy) team led to a similar comment.

lumba [6] (#numba), on the other hand, suggested that they would be interested in ne proposed change in a post-1.0 future for themselves [7] (#numba-interest).

he experimental Coconut JIT [13] (#coconut) could have benefitted from this PEP. ı private conversations with Coconut's creator we were told that our API was robably superior to the one they developed for Coconut to add JIT support to :Python.

)ebugging (#debugging)

ı conversations with the Python Tools for Visual Studio team (PTVS) [12] (#ptvs), ney thought they would find these API changes useful for implementing more erformant debugging. As mentioned in the Rationale (#rationale) section, this API /ould allow for switching on debugging functionality only in frames where it is eeded. This could allow for either skipping information that `sys.settrace()` ormally provides and even go as far as to dynamically rewrite bytecode prior to xecution to inject e.g. breakpoints in the bytecode.

also turns out that Google provides a very similar API internally. It has been used or performant debugging purposes.

mplementation (#implementation)

 set of patches implementing the proposed API is available through the Pyjion roject [1] (#id21). In its current form it has more changes to CPython than just this roposed API, but that is for ease of development instead of strict requirements to ccomplish its goals.

)pen Issues (#open-issues)

llow `eval_frame` to be `NULL` (#allow-eval-frame-to-be-null)

urrently the frame evaluation function is expected to always be set. It could very asily simply default to `NULL` instead which would signal to use `PyEval_EvalFrameDefault()`. The current proposal of not special-casing the field eemed the most straightforward, but it does require that the field not accidentally e cleared, else a crash may occur.

ejected Ideas (#rejected-ideas)

JIT-specific C API (#a-jit-specific-c-api)

riginally this PEP was going to propose a much larger API change which was ore JIT-specific. After soliciting feedback from the Numba team [6] (#numba), ough, it became clear that the API was unnecessarily large. The realization was ade that all that was truly needed was the opportunity to provide a trampoline unction to handle execution of Python code that had been JIT-compiled and a ay to attach that compiled machine code along with other critical data to the orresponding Python code object. Once it was shown that there was no loss in unctionality or in performance while minimizing the API changes required, the roposal was changed to its current form.

s co_extra needed? (#is-co-extra-needed)

hile discussing this PEP at PyCon US 2016, some core developers expressed eir worry of the `co_extra` field making code objects mutable. The thinking eemed to be that having a field that was mutated after the creation of the code

bject made the object seem mutable, even though no other aspect of code bjects changed.

he view of this PEP is that the `co_extra` field doesn't change the fact that code bjects are immutable. The field is specified in this PEP to not contain information equired to make the code object usable, making it more of a caching field. It ould be viewed as similar to the UTF-8 cache that string objects have internally; trings are still considered immutable even though they have a field that is onditionally set.

erformance measurements were also made where the field was not available for IT workloads. The loss of the field was deemed too costly to performance when sing an unordered map from C++ or Python's dict to associated a code object ith JIT-specific data objects.

eferences (#references)

l] *(1 (#id11), 2 (#id19))*

Pyjion project (https://github.com/microsoft/pyjion (https://github.com/microsoft/pyjion))

2] *(1 (#id1), 2 (#id4))*

CPython's C API (https://docs.python.org/3/c-api/index.html (https://docs.python.org/3/c-api/index.html))

3] *(1 (#id3), 2 (#id6))*

`PyCodeObject` (https://docs.python.org/3/c-api/code.html#c.PyCodeObject (https://docs.python.org/3/c-api/code.html#c.PyCodeObject))

4 (#id12)]

.NET Core Runtime (CoreCLR) (https://github.com/dotnet/coreclr (https://github.com/dotnet/coreclr))

5] *(1 (#id2), 2 (#id5), 3 (#id7))*

`PyEval_EvalFrameEx()` (https://docs.python.org/3/c-api/veryhigh.html?highlight=pyframeobject#c.PyEval_EvalFrameEx (https://docs.python.org/3/c-api/veryhigh.html?highlight=pyframeobject#c.PyEval_EvalFrameEx))

6] *(1 (#id15), 2 (#id20))*

Numba (http://numba.pydata.org/ (http://numba.pydata.org/))

7 (#id16)]

numba-users mailing list: “Would the C API for a JIT entrypoint being proposed by Pyjion help out Numba?” (https://groups.google.com/a/continuum.io/forum/#!topic/numba-users/yRl_0t8-m1g (https://groups.google.com/a/continuum.io/forum/#!topic/numba-users/yRl_0t8-m1g))

3 (#id10)]

[Python-Dev] Opcode cache in ceval loop (https://mail.python.org/pipermail/python-dev/2016-February/143025.html (https://mail.python.org/pipermail/python-dev/2016-February/143025.html))

9 (#id9)]

Python benchmark suite (https://hg.python.org/benchmarks (https://hg.python.org/benchmarks))

10 (#id13)]

Pyston (http://pyston.org (http://pyston.org))

11 (#id14)]

PyPy (http://pypy.org/ (http://pypy.org/))

12 (#id18)]

Python Tools for Visual Studio (http://microsoft.github.io/PTVS/ (http://microsoft.github.io/PTVS/))

13 (#id17)]

Coconut (https://github.com/davidmalcolm/coconut (https://github.com/davidmalcolm/coconut))

14 (#id8)]

pybench (https://hg.python.org/cpython/file/default/Tools/pybench (https://hg.python.org/cpython/file/default/Tools/pybench))

Copyright (#copyright)

This document has been placed in the public domain.